

Combining Discrete Math with Theory of Computation

ASHWIN LALL, DENISON UNIVERSITY

This article demonstrates how Denison University's Computer Science (CS) program combined its discrete math and theory of computation courses into a single course. This combination has the advantage of including theory of computation in the core without increasing the number of courses needed in the major. It also helps to motivate students to study discrete math topics with applications from the theory of computation.

Introduction

With increasing demands on what should be included in the CS curriculum, CS programs find themselves lacking the resources to teach theory of computation, arguably the very foundation of our discipline. In a recent survey of 500 CS Bachelor of Science programs, it was found that while 88% of them require discrete math, only 38% require a

course on computation theory [1]. This has been identified as one of ten “wicked” issues facing CS curricular design [2]. Simultaneously, it can be challenging to convey to students the role discrete math has in the discipline. For example, there has been a recent attempt to blend discrete math with programming [8]. To address both issues, our program at Denison University has combined its discrete math course with one on the basic theory of computation. This article shares some details about this approach, including advantages and disadvantages.

Course Details

Since 2016, our program has offered a course that teaches students discrete math just in time to be able to apply the concepts to automata theory and the theory of computation. This allowed us to introduce the basic theory

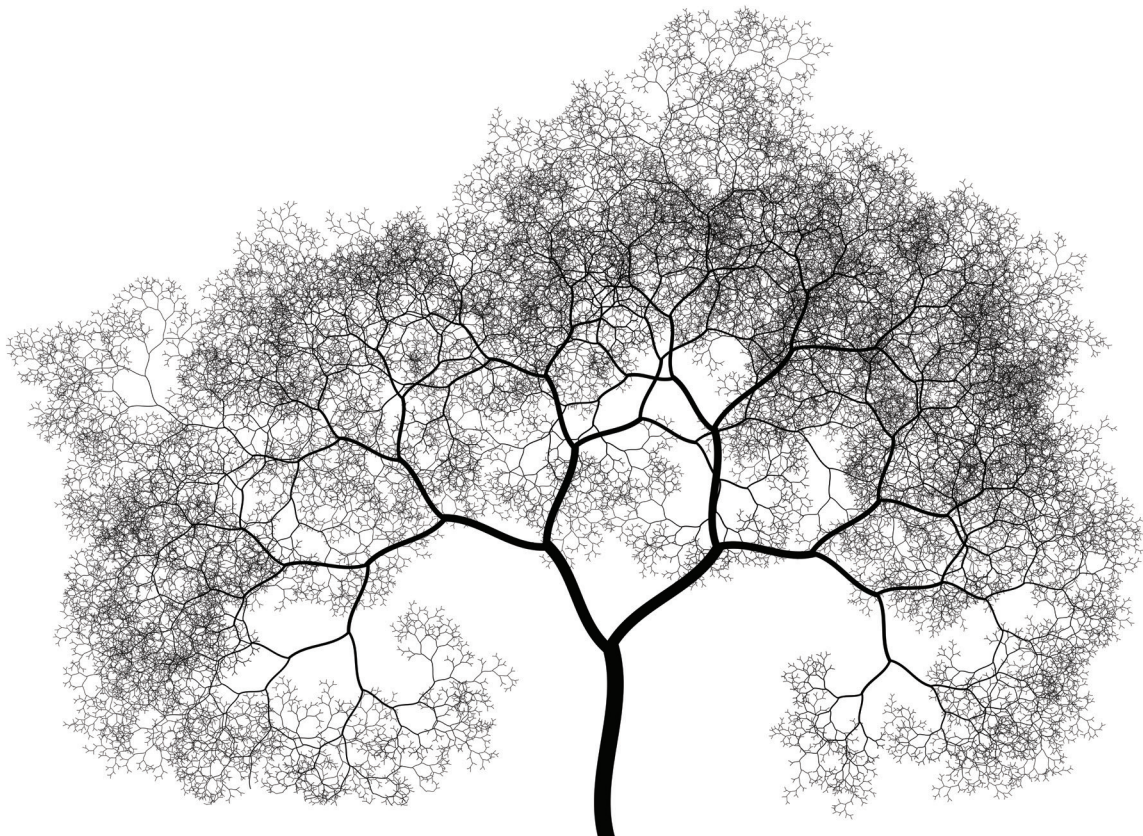


ILLUSTRATION: © SHUTTERSTOCK/SYNTHETICK

In a recent survey of 500 CS Bachelor of Science programs, it was found that while 88% of them require discrete math, only 38% require a course on computation theory.

of computation at the core of our curriculum rather than as an elective not seen by all our majors. This combination is especially effective at credit-limited institutions such as liberal arts colleges as it doesn't increase the number of credits required for a CS major. The prerequisites for the course are CS1 (including recursion) and some algebra (high school or college), so it can be taken early in the major.

This course lends itself to making easy connections between discrete math and the theory of computation, such as connecting sets and strings to languages of deterministic finite automata (DFA), direct proof with closure properties for regular languages, proof by contradiction with the pumping lemma, and induction with proving the language of a DFA. These connections are summarized in Figure 1. Of course, it should be expected that so much discrete math is needed for learning the theory of computation; the novelty here is using the latter to motivate the former.

Perhaps one of the most surprising aspects of this combined course is that it is indeed possible to teach virtually all the discrete math topics in CS2023 while infusing theory of computation topics, including deterministic and non-deterministic finite automata, regular expressions, closure properties of regular languages, the pumping lemma for regular languages, Turing machines, computability, and undecidability [4]. This is made possible by simply timing the introduction of theory of computation applications so they are used to solidify the discrete math topics. Of course, some topics (asymptotic notation, graphs, counting) do not

directly fit with this approach and must be taught separately at various points. A schedule for a 15-week semester is provided in Table 1.

Table 1. Weekly topics for the course

Week	Topic
1	Sets and languages
2	Finite automata
3	Logic and asymptotic notation
4	Nondeterministic automata
5	Regular expressions
6	Direct proof and closure properties
7	Proof by induction
8	Loop variants and strong induction
9	Proof by contradiction
10	Pumping lemma
11	Turing machines
12	Recursive and recursively enumerable languages
13	Undecidability
14	Recurrences
15	Counting and probability

The ability to include basic theory of computation all the way from strings up to Turing machines, undecidability, and simple reductions in the same course has fundamentally changed what our students get to see in the core of the major. Previously, we taught these topics in our theory of computation elective, which meant we were graduating some CS majors who didn't see any theory of computation. The change resolved our dissatisfaction that some of our CS majors graduated without learning about Turing machines and the Halting Problem.

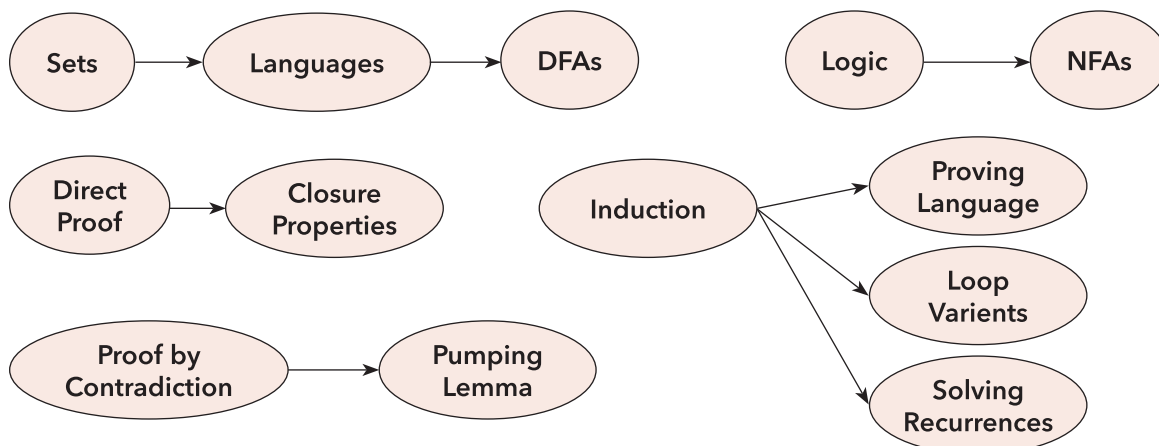


Figure 1. Discrete math and theory of computation connections.

The biggest advantage of this course is it allows programs with separate discrete math and automata theory courses to merge into a single course to free up a curricular spot for other topics needed at the core of the curriculum.

For the first few years of this course, we used a combination of automata theory [7] and math proof [3] textbooks to teach the course. However, this approach did not emphasize the close connections or get good coverage of topics such as asymptotic notation and graphs. More recently, we have been using a book, written by the author of this article, that introduces discrete math topics in the context of automata theory and computability [5]. The website [6] for this book has the full PDF as well as other resources including suggested homework questions.

Advantages

The biggest advantage of this course is it allows programs with separate discrete math and automata theory courses to merge into a single course to free up a curricular spot for other topics needed at the core of the curriculum. Alternatively, it allows programs that do not have any automata theory and Turing machine topics in the core of the curriculum to include these topics. Second, students get to learn discrete math in the context of computer science topics. By introducing each math topic just in time to learn how it applies to a computer science topic, students can see the relevance of math to the discipline of computer science and to their future courses. For example, we can motivate induction by showing them that it is intimately connected to why recursive functions work correctly. Similarly, we can show students how sets and strings are connected to automata theory rather than just studying them for their own sake. Any lesson on quantifiers can soon be followed up with a discussion of how they are used in Big-O notation.

Disadvantages

As a result of the blend in courses, there is less coverage of theory-of-computation topics (e.g., pushdown automata,

grammars, time and space complexity, equivalent models of computation). These can be covered in a later theory of computation elective. This leads to the second challenge: Any later theory-of-computation course needs to be modified to start from where this course leaves off. Decisions will have to be made about whether to circle back to teach about pushdown automata and other topics. On the positive side, once students have seen the basics of automata theory and Turing machines, the theory of computation course could start at a much later point (e.g., Turing machines and reductions) and thus examine deeper topics in the theory of computation than can usually be tackled in a single semester.

Summary

Combining discrete math with basic theory of computation has shown itself to be an effective way to compress two important topics into a single course to free up space in the CS curriculum at Denison University. Moreover, the use of CS applications from the theory of computation can provide a stronger motivation for students. They can also be exposed to the theoretical underpinnings of computation and uncomputability far earlier than would otherwise be possible. There are resources available for those looking to try out this approach [6]. ♦

REFERENCES

1. Blumenthal, R. Alignment among normative, prescriptive, and descriptive models of computer science curriculum: The effect of ABET accreditation on CS education. *ACM Transactions on Computing Education* 22, 3 (2022), 1–27.
2. Blumenthal, R. and Blumenthal, J. Moving what's in the CS curriculum forward: A proposition to address ten wicked curricular issues. In *Proceedings of the 56th ACM Technical Symp. on Computer Science Education* 1 Association for Computing Machinery, (2025), 137–143.
3. Hammack, R. *Book of Proof*, 3rd ed. (2018).
4. Kumar, A.N. et al. Computer Science Curricula 2023. Association for Computing Machinery (2024).
5. Lall, A. *Mathematical Foundations of Computer Science*. CRC Press (2024).
6. Lall, A. *Mathematical Foundations of Computer Science*; <https://personal.denison.edu/~lalla/MFCS> (2025).
7. Linz, P. *An Introduction to Formal Language and Automata*. Jones and Bartlett Publishers, Inc., USA (2006).
8. Liu, Y.A. and Castellana, M. Discrete math with programming: A principled approach. In *Proceedings of the 52nd ACM Technical Symp. on Computer Science Education* Association for Computing Machinery (2021), 1156–1162.

ASHWIN LALL

Department of Computer Science
Denison University
Granville, OH, USA
lalla@denison.edu

© 2026 Copyright held by the owner/author(s). DOI: 10.1145/3785011